INDEXX.AI PRODUCT DEVELOPMENT

Yay-chat AI Development Milestone Plan

Controlled Implementation Roadmap

YAY-CHAT

Yay-chat

AI, Messaging, Communities, Rewards, Wallet, and Ecosystem Integrations

Implementation instructions organized into ten controlled development milestones.

Table of Contents

Document Purpose 3

1. Product Direction 4
- 1.1 Product Name . . . 4
- 1.2 Product Role . . . 4
- 1.3 Core Product Formula . . . 4
- 1.4 Founder Vision . . . 4
- 1.5 X-to-Earn Philosophy . . . 4

2. Global Development Rules 6
- 2.1 Do Not Clone Existing Products . . . 6
- 2.2 Existing Repository Usage . . . 6
- 2.3 New Product Identity . . . 6
- 2.4 Implementation Discipline . . . 7

3. Milestone Overview 8

4. Milestone 1 — Complete Frontend Experience 9
- 4.1 Objective . . . 9
- 4.2 Scope . . . 9
- 4.3 Frontend Technical Requirements . . . 9
- 4.4 Required Design System . . . 10
- 4.5 Required Navigation . . . 10
- 4.6 Required Frontend Screens . . . 11
- 4.7 Mock Data and Mock Services . . . 15
- 4.8 Frontend Deliverables . . . 15
- 4.9 Milestone 1 Acceptance Criteria . . . 16

5. Milestone 2 — Backend and Platform Foundation 17
- 5.1 Objective . . . 17
- 5.2 Scope . . . 17
- 5.3 Required Technical Decisions . . . 17
- 5.4 Deliverables . . . 17
- 5.5 Acceptance Criteria . . . 18

6. Milestone 3 — Real-Time Messaging 19
- 6.1 Objective . . . 19
- 6.2 Existing Repository Audit . . . 19
- 6.3 Scope . . . 19
- 6.4 Acceptance Criteria . . . 20

7. Milestone 4 — Communities and Moderation 21
- 7.1 Objective . . . 21
- 7.2 Scope . . . 21
- 7.3 Acceptance Criteria . . . 21

8. Milestone 5 — AI Assistant and AI Features 22
- 8.1 Objective . . . 22
- 8.2 Scope . . . 22
- 8.3 Important Rules . . . 22
- 8.4 Acceptance Criteria . . . 22

9. Milestone 6 — Rewards and X-to-Earn Foundation 23
- 9.1 Objective . . . 23
- 9.2 Scope . . . 23

9.3 Required Decisions . . . 23
9.4 Acceptance Criteria . . . 23

10. Milestone 7 — BTCY and Wallet Foundation 24
10.1 Objective . . . 24
10.2 Required Decisions Before Development . . . 24
10.3 Scope . . . 24
10.4 Acceptance Criteria . . . 24

11. Milestone 8 — Indexx Product Integrations 25
11.1 Objective . . . 25
11.2 Recommended Integration Order . . . 25
11.3 Integration Contract . . . 25
11.4 Implementation Options . . . 25
11.5 Acceptance Criteria . . . 25

12. Milestone 9 — Merchant, Advertising, and Growth 27
12.1 Objective . . . 27
12.2 Scope . . . 27
12.3 Acceptance Criteria . . . 27

13. Milestone 10 — Production Hardening and Launch 28
13.1 Objective . . . 28
13.2 Scope . . . 28
13.3 Acceptance Criteria . . . 28

14. WeChat Feature Benchmark Document 29

15. Required Project Documentation 31

16. AI Developer Output Format for Every Milestone 32

17. First Instruction to the AI Developer 33

Document Purpose

This document is the implementation instruction for developing **Yay-chat** in controlled milestones.

Yay-chat is a new Indexx.ai product. It must not be treated as a renamed copy of Bitcoin Yay or a visual copy of WeChat.

The existing repository:

`Indexx-Bitcoin-2-0/bitcoin-yay-mobile`

may be inspected only to understand and selectively reuse chat-related logic. The full application, branding, navigation, wallet, mining screens, dashboard, visual design, and unrelated product code must not be copied into Yay-chat.

The main product source of truth is:

`yaychat_unified_product_vision_prd.md`

When an earlier instruction conflicts with that document, the unified PRD takes priority.

---

1. Product Direction

1.1 Product Name

Yay-chat

1.2 Product Role

Yay-chat is intended to become the main daily entry point into the Indexx ecosystem.

Chat is the starting experience, but the long-term platform also includes:

- Messaging
- Communities
- AI
- Rewards
- Wallet
- BTCY
- ShopperPal
- ReHuman
- EMMM
- Exchange
- Merchant tools
- Advertising
- Future Indexx products

1.3 Core Product Formula

```
Strong communication foundation
+
Original Yay-chat interface
+
AI
+
Communities
+
X-to-Earn rewards
+
Wallet and crypto capabilities
+
Indexx product integrations
=
Yay-chat
```

1.4 Founder Vision

The desired user behavior is:

> *Every morning, a user should open Yay-chat first.*

The application should not rely only on messaging for retention. It should create daily utility through communication, AI, communities, rewards, and access to the wider Indexx ecosystem.

1.5 X-to-Earn Philosophy

The platform should eventually support activity-based reward models such as:

- Chat to Earn
- Shop to Earn

- Mine to Earn
- Order to Earn
- Learn to Earn
- AI to Earn
- Group to Earn
- Post to Earn
- Watch Ads to Earn
- Invite Friends to Earn
- Open App to Earn
- Bet or Predict to Earn

Reward features must later be supported by a proper reward engine, audit ledger, eligibility rules, limits, reversals, campaign controls, and anti-fraud measures.

---

2. Global Development Rules

The AI developer must follow these rules throughout all milestones.

2.1 Do Not Clone Existing Products

Do not copy the full Bitcoin Yay application.

Do not copy the visual interface of:

- Bitcoin Yay
- WeChat
- WhatsApp
- Telegram
- Any other existing messaging application

Familiar usability patterns may be used, but the final product must have its own identity.

2.2 Existing Repository Usage

The Bitcoin Yay mobile repository may only be used to inspect chat-related areas such as:

- Conversation models
- Message models
- Chat APIs
- Socket or WebSocket handling
- Message synchronization
- Media messages
- Voice notes
- Typing indicators
- Presence
- Read receipts
- Delivery status
- Replies
- Reactions
- Group chat
- Notifications
- Search
- Pagination
- Local caching

If useful chat code is tightly coupled to unrelated Bitcoin Yay functionality, recreate the behavior cleanly inside Yay-chat rather than copying unnecessary dependencies.

2.3 New Product Identity

Yay-chat must have:

- Its own application structure
- Its own package and bundle identifiers
- Its own design system
- Its own navigation
- Its own assets
- Its own colors and typography
- Its own environment configuration
- Its own release setup

2.4 Implementation Discipline

For every milestone:

1. Read the relevant sections of the unified PRD.
2. Document assumptions before implementing them.
3. Separate confirmed requirements from recommendations.
4. Avoid introducing unnecessary technologies.
5. Keep components modular and reusable.
6. Add tests for critical behavior.
7. Update documentation after implementation.
8. Do not claim incomplete or mocked functionality is production-ready.
9. Do not begin the next milestone until the current milestone meets its acceptance criteria.

---

3. Milestone Overview

Milestone	Name	Primary Outcome
1	Complete Frontend Experience	Full Yay-chat mobile UI, navigation, screens, interactions, mock data, and frontend tests
2	Backend and Platform Foundation	Authentication, profiles, shared APIs, database foundations, environments, logging, and deployment base
3	Real-Time Messaging	Production messaging, groups, media, voice notes, receipts, presence, notifications, and synchronization
4	Communities and Moderation	Public/private communities, roles, invites, announcements, polls, events, moderation, and discovery
5	AI Assistant and AI Features	AI chat, in-conversation assistance, translation, summaries, document support, and usage controls
6	Rewards and X-to-Earn Foundation	Reward events, rules engine, ledger, referrals, daily check-ins, limits, and anti-abuse controls
7	BTCY and Wallet Foundation	Limited BTCY integration, balances, transaction history, security model, and wallet readiness
8	Indexx Product Integrations	ShopperPal, ReHuman, EMMM, Exchange, and future product connections
9	Merchant, Advertising, and Growth	Merchant accounts, promotions, rewarded ads, analytics, campaigns, and monetization tools
10	Production Hardening and Launch	Security, performance, accessibility, compliance, QA, observability, store readiness, and release

4. Milestone 1 — Complete Frontend Experience

4.1 Objective

Build the complete frontend of Yay-chat before connecting production backend services.

At the end of this milestone, stakeholders must be able to install and navigate a polished mobile application that demonstrates the full intended user experience using mock data and mocked service responses.

The frontend must be original and must not visually resemble a renamed Bitcoin Yay application.

4.2 Scope

Milestone 1 includes:

- Product sitemap
- Navigation architecture
- Original visual direction
- Design system
- Reusable UI components
- Complete screen inventory
- Primary user flows
- All main screens
- Loading, empty, error, offline, and success states
- Mocked API layer
- Mock data
- Frontend state management
- Local interaction behavior
- Accessibility basics
- Dark mode if approved by the design direction
- Frontend unit and interaction tests
- Frontend documentation

Milestone 1 does not include production backend integration, real crypto transactions, real rewards, real payment processing, or production AI calls.

4.3 Frontend Technical Requirements

Use the existing mobile technology only if it is suitable for the new product. React Native may be used if it matches the current team and project environment.

The frontend must include:

- Type-safe application code
- Clear folder structure
- Reusable design tokens
- Reusable components
- Centralized navigation definitions
- Centralized route types
- Mock service adapters
- Environment-based configuration
- Form validation
- Error boundaries
- Retry patterns
- Local caching strategy
- Secure token storage interface, even if mocked initially
- Analytics event interface, even if mocked initially
- Feature-flag interface
- Testable state management

Do not connect screens directly to hardcoded data. Use a mock service layer that can later be replaced with real APIs.

4.4 Required Design System

Create an original Yay-chat design system containing:

- Brand color tokens
- Neutral colors
- Semantic colors
- Typography scale
- Font weights
- Spacing scale
- Border-radius scale
- Elevation and shadows
- Icon rules
- Buttons
- Text inputs
- Search inputs
- Checkboxes
- Radio controls
- Switches
- Tabs
- Chips
- Badges
- Avatars
- Cards
- List rows
- Message bubbles
- Composer controls
- Bottom sheets
- Modals
- Toasts
- Banners
- Empty states
- Skeleton loaders
- Error states
- Offline states
- Confirmation states

Document each component and its supported states.

4.5 Required Navigation

Design and implement a scalable navigation structure.

The initial primary navigation should evaluate the following areas:

- Chats
- Communities
- AI
- Earn
- Profile

Additional modules such as Wallet, BTCY, ShopperPal, ReHuman, EMMM, Exchange, and future services should not overload the main navigation. Use progressive disclosure and contextual entry points.

The final navigation must be documented with:

- Root navigation

- Authentication navigation
- Main tab navigation
- Nested stack navigation
- Modal routes
- Deep-link strategy
- Future module entry points

4.6 Required Frontend Screens

A. Onboarding and Authentication

Create complete UI and mocked flows for:

- Splash screen
- Welcome screen
- Sign in
- Sign up
- Email verification
- Phone verification
- Forgot password
- Reset password
- Terms acceptance
- Privacy acceptance
- Username selection
- Profile setup
- Avatar setup
- Permission requests
- Notification permission explanation
- Contact permission explanation
- Onboarding completion

B. Chats

Create complete UI and mocked flows for:

- Chat list
- Search conversations
- Filter conversations
- Archived chats
- New chat
- Contact selection
- One-to-one conversation
- Group conversation
- Message composer
- Text messages
- Emoji
- Stickers placeholder
- GIF placeholder
- Image messages
- Video messages
- File messages
- Voice notes
- Replies
- Reactions
- Mentions
- Message forwarding
- Message deletion
- Message recall state
- Copy message

- Pin message
- Delivery status
- Read status
- Typing indicator
- Online presence
- Failed message state
- Retry sending
- Upload progress
- Download progress
- Date separators
- Unread message separators
- Search inside conversation
- Conversation details
- Mute conversation
- Block user
- Report user
- Shared media
- Group members
- Group settings
- Group roles
- Leave group

C. Communities

Create complete UI and mocked flows for:

- Community discovery
- Community search
- Community categories
- Public community page
- Private community page
- Join request
- Invite-only state
- Community feed
- Announcements
- Community chat
- Community members
- Admin and moderator views
- Community rules
- Community events
- Polls
- Invite links
- Report community
- Leave community
- Create community
- Edit community

D. AI

Create complete UI and mocked flows for:

- AI home
- Start AI conversation
- AI chat
- Suggested prompts
- Ask a question
- Summarize text
- Summarize PDF placeholder
- Translate content

- Write an email
- Study assistant
- Coding assistant
- Financial assistant disclaimer state
- Image generation placeholder
- AI history
- Saved AI conversations
- AI usage indicator
- AI plan or credits placeholder
- AI error state
- AI unavailable state

E. Earn and Rewards

Create complete UI and mocked flows for:

- Earn dashboard
- Daily check-in
- Current streak
- Available activities
- Chat to Earn
- Invite Friends to Earn
- AI to Earn
- Group to Earn
- Shop to Earn placeholder
- Mine to Earn placeholder
- Watch Ads to Earn placeholder
- Reward history
- Reward detail
- Pending reward
- Completed reward
- Reversed reward
- Reward limits
- Campaign detail
- Referral dashboard
- Referral code
- Referral sharing
- Anti-abuse warning state

Clearly label mocked balances and test rewards in the development build.

F. Wallet and BTCY Preview

Create frontend-only preview screens for:

- Wallet overview
- Asset list
- BTCY balance
- Nugget balance
- Transaction history
- Transaction detail
- Send preview
- Receive preview
- QR receive preview
- Security warning
- Wallet setup placeholder
- Wallet unavailable state

These screens must not imply that real financial transactions are active.

G. Ecosystem Discovery

Create UI entry points and preview states for:

- BTCY
- ShopperPal
- ReHuman
- EMMM
- Exchange
- Future Indexx products

Each product preview should show:

- Product purpose
- User benefit
- Relevant X-to-Earn action
- Current availability
- Coming-soon state where applicable
- Future integration entry point

H. Profile and Settings

Create complete UI and mocked flows for:

- User profile
- Edit profile
- QR profile
- Friends or contacts
- Blocked users
- Privacy settings
- Notification settings
- Chat settings
- Community settings
- AI settings
- Rewards settings
- Wallet security settings placeholder
- Device management
- Active sessions
- Data and storage
- Language
- Appearance
- Accessibility
- Help center
- Contact support
- Report a problem
- Terms
- Privacy policy
- Delete account
- Sign out

I. Global States

Create reusable screens and components for:

- No internet
- Service unavailable
- Maintenance
- Session expired
- Permission denied
- Content unavailable
- Empty list

- Search with no results
- Loading
- Skeleton loading
- Partial loading
- Error with retry
- Success confirmation
- Account restricted
- Feature unavailable by region
- Feature coming soon

4.7 Mock Data and Mock Services

Create a realistic mock service layer for:

- Authentication
- Profiles
- Conversations
- Messages
- Groups
- Communities
- AI conversations
- Rewards
- Referrals
- Wallet previews
- BTCY previews
- Product discovery
- Notifications
- Search
- Settings

The mock layer must support:

- Success responses
- Delayed responses
- Empty responses
- Validation errors
- Authorization errors
- Server errors
- Offline behavior
- Pagination
- Retry behavior

4.8 Frontend Deliverables

Create and maintain:

- `docs/yaychat-product-sitemap.md`
- `docs/yaychat-navigation-map.md`
- `docs/yaychat-screen-inventory.md`
- `docs/yaychat-user-flows.md`
- `docs/yaychat-ui-system.md`
- `docs/yaychat-component-inventory.md`
- `docs/yaychat-frontend-architecture.md`
- `docs/yaychat-mock-api-contracts.md`
- `docs/yaychat-frontend-test-plan.md`

4.9 Milestone 1 Acceptance Criteria

Milestone 1 is complete only when:

- The application has original Yay-chat branding.
- No Bitcoin Yay visual identity remains.
- All listed frontend areas are implemented or clearly marked as approved placeholders.
- All primary user journeys can be completed using mock services.
- Navigation works consistently.
- Loading, empty, error, retry, offline, and success states exist.
- Components are reusable and documented.
- Mock services are separated from UI components.
- Critical frontend flows have automated tests.
- The application works on supported iOS and Android development environments.
- No screen claims that mocked wallet, AI, reward, or crypto behavior is real.
- Stakeholders can review the complete product experience without a production backend.
- The frontend architecture is ready for real API integration in Milestone 2 and Milestone 3.

---

5. Milestone 2 — Backend and Platform Foundation

5.1 Objective

Create the secure backend foundation required by the completed frontend.

5.2 Scope

Implement:

- Environment configuration
- API gateway or backend entry layer
- Authentication
- Session management
- User profiles
- Device registration
- Role and permission foundation
- Database setup
- Media storage foundation
- Notification foundation
- Logging
- Monitoring
- Error reporting
- Audit logging
- Feature flags
- Analytics ingestion foundation
- Admin access foundation
- CI/CD foundation
- Development, staging, and production environments

5.3 Required Technical Decisions

Document and approve:

- Backend framework
- Primary database
- Cache
- Object storage
- Authentication method
- Token strategy
- Session revocation
- API versioning
- Rate limiting
- Secrets management
- Logging strategy
- Monitoring strategy
- Deployment model
- Backup and recovery

Do not assume that one database technology should handle all workloads.

Financial, reward, permission, and audit records require strong consistency and traceability.

5.4 Deliverables

- `docs/yaychat-architecture.md`
- `docs/yaychat-data-models.md`
- `docs/yaychat-api-contracts.md`
- `docs/yaychat-authentication-and-sessions.md`

- `docs/yaychat-environments-and-deployment.md`
- `docs/yaychat-observability.md`

5.5 Acceptance Criteria

- Users can register, sign in, sign out, recover accounts, and manage sessions.
- Profiles are persisted.
- Frontend authentication and profile screens use real APIs.
- Authorization rules are enforced.
- Environments are separated.
- Logs, metrics, and error reporting are working.
- API contracts match the frontend service layer.
- Critical backend behavior has automated tests.

---

6. Milestone 3 — Real-Time Messaging

6.1 Objective

Replace mocked chat behavior with production-ready communication services.

6.2 Existing Repository Audit

Before implementation, inspect chat-related code in:

`Indexx-Bitcoin-2-0/bitcoin-yay-mobile`

Create:

`docs/yaychat-current-chat-audit.md`

The audit must identify:

- Existing chat capabilities
- Reusable logic
- Unnecessary dependencies
- Security concerns
- Performance concerns
- Missing functionality
- Recommended reuse
- Recommended rebuilds

6.3 Scope

Implement:

- One-to-one conversations
- Group conversations
- Conversation list
- Real-time message delivery
- Message history
- Pagination
- Message ordering
- Message retries
- Idempotency
- Delivery receipts
- Read receipts
- Typing indicators
- Presence
- Replies
- Reactions
- Mentions
- Forwarding
- Deletion or recall rules
- Pinned messages
- Media messages
- File uploads
- Voice notes
- Search
- Push notifications
- Offline synchronization
- Multi-device session behavior
- Blocking
- Reporting

6.4 Acceptance Criteria

- Two real users can communicate reliably.
- Group messaging works with roles and membership rules.
- Messages remain consistent across reconnects.
- Duplicate sends are prevented.
- Offline messages synchronize correctly.
- Media uploads are secure.
- Notifications open the correct conversation.
- Blocking and reporting work.
- Critical messaging flows have integration and end-to-end tests.

---

7. Milestone 4 — Communities and Moderation

7.1 Objective

Build the community layer that expands Yay-chat beyond direct messaging.

7.2 Scope

Implement:

- Public communities
- Private communities
- Community discovery
- Community search
- Join requests
- Invite links
- Roles
- Admins
- Moderators
- Members
- Announcements
- Community chats
- Polls
- Events
- Community rules
- Reporting
- Banning
- Content moderation
- Moderation queues
- Community analytics foundation

7.3 Acceptance Criteria

- Users can discover, join, leave, and participate in communities.
- Private access rules work.
- Admin and moderator permissions are enforced.
- Reports enter a moderation workflow.
- Community content is searchable where permitted.
- Critical permission scenarios have automated tests.

---

8. Milestone 5 — AI Assistant and AI Features

8.1 Objective

Make AI a daily utility inside Yay-chat.

8.2 Scope

Implement:

- General AI assistant
- AI conversation history
- Suggested prompts
- Message translation
- Conversation summarization
- Community summarization
- Email writing
- Study assistance
- Coding assistance
- Document summarization
- Image generation integration if approved
- AI inside chats
- AI inside communities
- Usage limits
- Credits or plan controls
- Safety filters
- Prompt and response logging rules
- Privacy controls
- Cost monitoring

8.3 Important Rules

- Do not send private chat content to AI without explicit user action and clear disclosure.
- Define data retention rules.
- Define which AI providers are used.
- Add rate limits and cost controls.
- Add appropriate disclaimers for financial, legal, medical, and other high-risk topics.

8.4 Acceptance Criteria

- AI functions are connected to real services.
- User consent and privacy controls are visible.
- Usage limits are enforced.
- Errors and provider outages are handled.
- Cost and token usage are observable.
- AI outputs can be reported.
- Critical AI workflows have automated tests.

---

9. Milestone 6 — Rewards and X-to-Earn Foundation

9.1 Objective

Create a centralized, auditable rewards platform rather than isolated reward screens.

9.2 Scope

Implement:

- Standard reward event model
- Reward rules engine
- Reward ledger
- Reward balances
- Pending rewards
- Completed rewards
- Reversed rewards
- Daily check-ins
- Referral rewards
- Chat activity rewards if approved
- AI activity rewards if approved
- Community contribution rewards if approved
- Campaigns
- Eligibility rules
- Limits
- Cooldowns
- Fraud detection foundation
- Manual review
- Admin adjustments
- Audit history
- User reward history

9.3 Required Decisions

Confirm:

- What users earn
- Whether rewards are points, nuggets, BTCY, or another unit
- Conversion rules
- Funding model
- Expiration rules
- Reversal rules
- Regional restrictions
- Tax and legal considerations
- Anti-fraud policy

9.4 Acceptance Criteria

- Every reward is linked to a verifiable event.
- Duplicate rewards are prevented.
- Ledger records are immutable or fully auditable.
- Limits and cooldowns are enforced.
- Reversals are recorded.
- Admin changes are audited.
- Suspicious activity can be flagged.
- Reward screens use real backend data.

---

10. Milestone 7 — BTCY and Wallet Foundation

10.1 Objective

Introduce controlled BTCY and wallet capabilities only after security, custody, and compliance decisions are approved.

10.2 Required Decisions Before Development

Confirm:

- Custodial or non-custodial model
- Supported assets
- Supported countries
- Identity verification requirements
- Recovery model
- Key management
- Transaction approval rules
- Withdrawal rules
- Deposit rules
- Fees
- Fraud controls
- Compliance responsibilities
- Security review requirements

10.3 Scope

Depending on approved decisions, implement:

- Wallet setup
- Balance view
- BTCY balance
- Nugget balance
- Transaction history
- Transaction details
- Internal transfers
- Receive flow
- QR flow
- Security controls
- Device confirmation
- Transaction limits
- Wallet notifications
- Support and dispute flow

External transfers must not be enabled until all required reviews are complete.

10.4 Acceptance Criteria

- Wallet behavior matches the approved custody model.
- Transactions are auditable.
- Sensitive actions require appropriate verification.
- Balances are strongly consistent.
- Recovery procedures are tested.
- Security review findings are resolved.
- Compliance requirements are documented.

---

11. Milestone 8 — Indexx Product Integrations

11.1 Objective

Turn Yay-chat into the central gateway for the Indexx ecosystem.

11.2 Recommended Integration Order

1. BTCY
2. ShopperPal
3. ReHuman
4. EMMM
5. Exchange
6. Future Indexx products

The actual order should follow product readiness and approved business priority.

11.3 Integration Contract

Every product integration must define:

- Entry point
- Authentication
- Shared identity
- Permission scopes
- Data-sharing rules
- Deep links
- Navigation behavior
- Reward events
- Wallet behavior
- Notifications
- Analytics
- Error handling
- Customer support
- Availability by region
- Compliance constraints

11.4 Implementation Options

An integration may use:

- Native screens
- Shared APIs
- Embedded web experiences
- WebView
- Internal mini-app model
- External deep links

Do not assume that every product must become a mini app.

Choose the method that provides the best user experience, security, maintainability, and delivery speed.

11.5 Acceptance Criteria

- Users can access approved Indexx products from Yay-chat.
- Authentication is seamless and secure.
- Data-sharing permissions are clear.
- Product events can produce reward events where approved.
- Navigation back to Yay-chat is reliable.

- Integration failures are handled clearly.

---

12. Milestone 9 — Merchant, Advertising, and Growth

12.1 Objective

Add business, promotion, and revenue capabilities.

12.2 Scope

Implement as approved:

- Merchant accounts
- Merchant verification
- Merchant profiles
- Shops
- Product listings
- Promotions
- Coupons
- Cashback
- Sponsored communities
- Sponsored channels
- Sponsored posts
- Native ads
- Banner ads
- Rewarded ads
- Campaign targeting
- Campaign limits
- Ad reporting
- Merchant analytics
- Billing
- Fraud prevention
- User ad preferences
- Ad reporting and hiding

12.3 Acceptance Criteria

- Merchants can be verified and managed.
- Promotions and ads follow campaign rules.
- Rewarded ads cannot be repeatedly abused.
- Users can identify sponsored content.
- Reporting and moderation exist.
- Billing and campaign records are auditable.

---

13. Milestone 10 — Production Hardening and Launch

13.1 Objective

Prepare Yay-chat for secure public release.

13.2 Scope

Complete:

- Security audit
- Privacy review
- Compliance review
- Threat modeling
- Penetration testing
- Load testing
- Real-time messaging stress testing
- Database performance testing
- Media-storage testing
- Accessibility review
- Localization readiness
- Crash monitoring
- Metrics and alerts
- Incident-response plan
- Backup and restore test
- Disaster-recovery plan
- Support workflows
- Moderation operations
- Store assets
- Store policies
- Release signing
- Beta program
- Staged rollout
- Rollback plan

13.3 Acceptance Criteria

- No critical security issues remain.
- Core user journeys pass end-to-end tests.
- Performance targets are met.
- Monitoring and alerts are active.
- Backup and recovery are tested.
- Store requirements are satisfied.
- Support and incident procedures are documented.
- A rollback path exists.
- Release approval is recorded.

---

14. WeChat Feature Benchmark Document

Create and maintain:

`docs/yaychat-vs-wechat-feature-gap.md`

Use WeChat as a capability benchmark, not as a visual template.

The document should compare:

- WeChat capabilities
- Existing Bitcoin Yay chat capabilities
- Features included in the Yay-chat PRD
- Missing Yay-chat capabilities
- MVP priority
- Later-phase priority
- Complexity
- Dependencies
- Region or compliance limitations

Recommended table:

Category	WeChat Capability	Current Bitcoin Yay Chat	Planned in Yay-chat	Gap
Priority	**Milestone**	**Complexity**	**Dependencies**	**Notes**

Use the following statuses:

- Available
- Partially available
- Missing
- Requires verification
- Region-specific
- Not applicable

Use the following priorities:

- P0 — Foundation
- P1 — MVP
- P2 — Post-MVP
- P3 — Long-term
- Not planned

The comparison should cover:

- Messaging
- Calls
- Groups
- Communities
- Social content
- Public accounts
- AI
- Translation
- Wallet
- Payments
- Shopping
- Merchant services
- Mini programs
- Search
- QR discovery
- Moderation

- Privacy
- Device management
- Business tools
- Advertising

---

15. Required Project Documentation

Maintain the following documents throughout development:

- `docs/yaychat-vs-wechat-feature-gap.md`
- `docs/yaychat-current-chat-audit.md`
- `docs/yaychat-product-sitemap.md`
- `docs/yaychat-navigation-map.md`
- `docs/yaychat-screen-inventory.md`
- `docs/yaychat-user-flows.md`
- `docs/yaychat-ui-system.md`
- `docs/yaychat-component-inventory.md`
- `docs/yaychat-frontend-architecture.md`
- `docs/yaychat-architecture.md`
- `docs/yaychat-data-models.md`
- `docs/yaychat-api-contracts.md`
- `docs/yaychat-reward-engine.md`
- `docs/yaychat-security-and-compliance.md`
- `docs/yaychat-development-roadmap.md`
- `docs/yaychat-open-decisions.md`
- `docs/yaychat-test-strategy.md`
- `docs/yaychat-release-plan.md`

Each document must distinguish:

- Confirmed requirement
- Proposed implementation
- Assumption
- Open decision
- Future feature
- Compliance dependency

---

16. AI Developer Output Format for Every Milestone

At the start of each milestone, provide:

1. Milestone objective
2. Existing code or systems being inspected
3. Assumptions
4. Open decisions
5. Architecture or UI approach
6. Task breakdown
7. Files expected to be created or modified
8. Risks
9. Acceptance criteria

During implementation:

- Keep changes limited to the active milestone.
- Do not silently add unrelated features.
- Do not remove existing working functionality without documenting why.
- Keep the application runnable after each major change.
- Add tests alongside critical functionality.
- Update relevant documentation.

At the end of each milestone, provide:

1. Completed items
2. Remaining items
3. Files changed
4. Tests added
5. Test results
6. Known limitations
7. Open decisions
8. Security concerns
9. Deployment notes
10. Confirmation that acceptance criteria were met or a list of unmet criteria

Do not begin the next milestone until the current milestone has been reviewed and approved.

---

17. First Instruction to the AI Developer

Begin with **Milestone 1 — Complete Frontend Experience**.

Before writing production code:

1. Read `yaychat_unified_product_vision_prd.md` completely.
2. Inspect the existing project structure without copying the full Bitcoin Yay product.
3. Review the existing chat screens only to understand available behavior.
4. Create the product sitemap.
5. Create the navigation map.
6. Create the screen inventory.
7. Define the original Yay-chat design system.
8. Define mock API contracts.
9. Implement the complete frontend using mock services.
10. Add frontend tests and documentation.

The frontend must be complete enough for stakeholder review before backend implementation begins.

Do not start Milestone 2 until Milestone 1 has been approved.